

Building Customer Support AI Agents at 100M-User Scale: An Evaluation-Driven Framework

Aman Gupta
Nubank
Palo Alto, USA
aman.gupta@nubank.com.br

Kevin Rossell
Nubank
Mexico City, Mexico
kevin.rossell@nubank.com.br

Edesio Alcobaça
Nubank
São Paulo, Brazil
edesio.alcobaca@nubank.com.br

Jose Chrystian Lima Pacheco
Nubank
São Paulo, Brazil
chrystian.lima@nubank.com.br

Carolina Baptista de Lima*
Nubank
São Paulo, Brazil
carolina.bap.lima@gmail.com

Shao Tang
Nubank
Palo Alto, USA
tang.shao@nubank.com.br

Luiz Paulo Rabachini
Nubank
São Paulo, Brazil
luiz.rabachini@nubank.com.br

Luis Moneda
Nubank
São Paulo, Brazil
luis.moneda@nubank.com.br

Herbert Fei
Nubank
Palo Alto, USA
herbert.fei@nubank.com.br

Daniel Silva
Nubank
Palo Alto, USA
daniel.ribeiro@nubank.com.br

Rohan Ramanath
Nubank
Palo Alto, USA
rohan.ramanath@nubank.com.br

Abstract

The rapid rise in LLM capabilities has made AI agents increasingly viable across a broad range of tasks. Among the most promising applications is building production-ready customer-facing agents—a challenge that demands coordinated excellence in evaluation methodology, context engineering, training, and online measurement. Yet these critical pillars are typically developed in isolation, creating blind spots that only surface after deployment.

In this paper, we present a unified framework that bridges offline development with online impact for customer support AI agents at Nubank - a company with 100m+ users. Our approach integrates several key components: (1) structured context engineering tailored to customer support agents (2) systematic human-in-the-loop prompt iteration, (3) rigorous LLM judge evaluation with measured inter-rater agreement and GEPA-optimization for consistency and (4) ideation-to-production validation.

A central insight is that evaluation-pipeline quality directly determines iteration velocity. We present results from five production deployments spanning distinct domains—card delivery, debt management, credit-limit support, card management, and product explanation — that deliver consistent customer-satisfaction gains while substantially accelerating iteration. In our card-delivery deployment, large-scale A/B testing yields a 37 percentage-point improvement in AI transactional Net Promoter Score and a 29

percentage-point gain in self-service rate over prior agent variants, alongside a strong correlation between offline simulation metrics and online outcomes—demonstrating that eval-driven development reliably predicts production impact. On most use cases, AI satisfaction reaches within a few percentage points of expert human agents.

CCS Concepts

• Computing methodologies → Natural language generation.

Keywords

customer support agents, LLM-as-a-judge, evaluation-driven development, prompt optimization, GEPA, inter-rater reliability

ACM Reference Format:

Aman Gupta, Kevin Rossell, Edesio Alcobaça, Jose Chrystian Lima Pacheco, Carolina Baptista de Lima, Shao Tang, Luiz Paulo Rabachini, Luis Moneda, Herbert Fei, Daniel Silva, and Rohan Ramanath. 2026. Building Customer Support AI Agents at 100M-User Scale: An Evaluation-Driven Framework. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3818332>

1 Introduction

The rise of large language models (LLMs) [5, 8, 9, 18, 19] has catalyzed a new paradigm: AI agents capable of dynamically directing their own processes, leveraging tools, and maintaining control over task execution [2, 11, 20, 45]. When equipped with the *right context and tools*, AI Agents have demonstrated impressive capabilities in solving difficult and long running tasks [6, 10, 24].

While AI agents have seen rapid success in coding and deep research [3, 16, 32], their potential in many other domains remains largely untapped. In **customer support (CS)**, they could transform

*Work done while at Nubank.



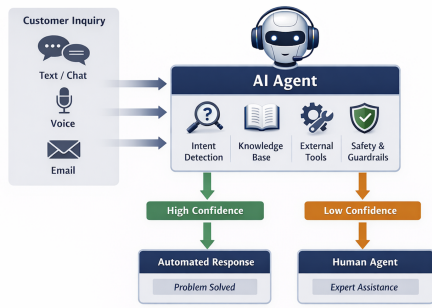


Figure 1: Architecture of a Customer Support AI agent system. User queries arrive via multiple channels and are handled autonomously, with fallback to human agents under low confidence.

the way organizations handle millions of interactions — evolving from scripted chatbots to systems that personalize experiences, maintain rich context, take actions through tools, and resolve complex issues end-to-end. Crucially, they can also act as co-pilots for human agents, handling routine cases and enabling people to focus on the hardest, long-tail problems.

1.1 Unique challenge of building CS AI agents

Compared to general-purpose AI agents, CS agents impose distinct requirements:

- **Higher quality bar.** In CS, failure is costly: a single poor interaction can erode trust and drive churn. This creates a stricter reliability threshold than in many research or coding settings. In a banking environment, any mishap in actions can lead to significant amounts of losses in a single case.
- **Sensitive data handling.** CS agents operate over real customer data (e.g., balances, transactions, addresses), which introduces privacy, security, and regulatory constraints.
- **Narrower scope, deeper specialization.** CS agents typically use a small, domain-specific toolset (often 5–15 tools), but must handle substantial edge-case diversity within that scope.
- **Graceful handoff to humans.** CS agents work within a human-in-the-loop workflow. When automation falls short, they must transfer control with full context preserved, since customer satisfaction reflects the end-to-end journey—not just the automated segment (for a high-level architecture, see **Figure 1**).

1.2 Our Contributions

We present a highly-scalable, production-tested design for building CS AI agents. Our design is based on evaluation and measurement, working backward from delivering a great customer experience (**Figure 2**). We validate our design on real-world CS use cases for a company that serves 100M+ users. Our contributions can be summarized as follows:

- **A unified, evaluation-driven framework** for developing production CS agents, connecting agent architecture, prompt versioning, offline evaluation, and online measurement in a closed

loop (**Figure 2**). A central insight is that evaluation pipeline quality directly determines iteration velocity.

- **Modular context engineering.** We propose a prompt architecture in which each component—instructions, routines, macros, tool descriptions, and working memory—is independently versioned, enabling controlled evolution and reuse (**Figure 4**).
- **A rigorous evaluation pipeline.** We introduce a calibrated LLM-as-Judge pipeline with (a) inter-rater reliability requirements enforcing minimum human-agreement thresholds before trusting automated judgments, (b) GEPA-optimized [1] judge prompts that align with human criteria, and (c) cross-model validation showing that well-specified rubrics yield stable scores across model families, sizes, and vendors.
- **Prod-validated results at scale across five use cases.** We report A/B-tested deployments of five production agents built with the same framework, spanning card delivery, debt management, credit-limit support, card management, and product explanation (**Figure 3**). For our headline card-delivery deployment, we achieve a **37 p.p.** absolute improvement in AI transactional net promoter score (tNPS - a measure of quality) and **29 p.p.** in self-service rate (SSR - a measure of automation) when compared to previous agent variants; the final tNPS is only 10 p.p. lower than expert human-agent tNPS. The four further agents show consistent customer-satisfaction gains in their respective domains (Section 6.4). Across most of these deployments the AI agent’s tNPS lands within a few percentage points of expert human agents (within ~1–10 p.p. for card delivery, credit-limit, card management, and product explanation), with a larger gap remaining in the hardest domain, debt management. Crucially, we demonstrate strong correlation between offline eval scores and online business metrics (**Figure 9**), validating that eval-driven development reliably predicts production impact.

2 Related Work

System Architectures and Benchmarks. Prior CS systems span modular designs - Obadinma et al. [31]’s Neural Agent Assistant (intent classification, retrieval, generation), Airbnb’s policy-to-schema assistant [39], and out-of-scope detection for graceful human hand-off [36]. On the data side, NatCS [17] offers a multi-domain support corpus, CSConv/RoleCS [48] a staged conversation framework with synthetic training data, and Mendonça et al. [29] a bilingual corpus annotated for emotion and dialogue quality.

Feedback, Trust, and Evaluation. Feng et al. [12] model user satisfaction via goal-tracking task slots; Følstad et al. [15] show early breakdowns erode trust; and Reinhard et al. [38] frame generative AI as augmenting human agents. Mohammadi et al. [30] survey LLM-agent evaluation by objective and mode (offline vs. online), noting enterprise challenges such as policy compliance and long-horizon reliability.

3 Background

3.1 AI Agents

While there is no consensus on the definition of *AI Agents*, we use it to specifically refer to the ReACT paradigm [45]. ReACT stands for Reason + Act, where an LLM interleaves chain-of-thought reasoning [42] with tool use and can autonomously work on a task

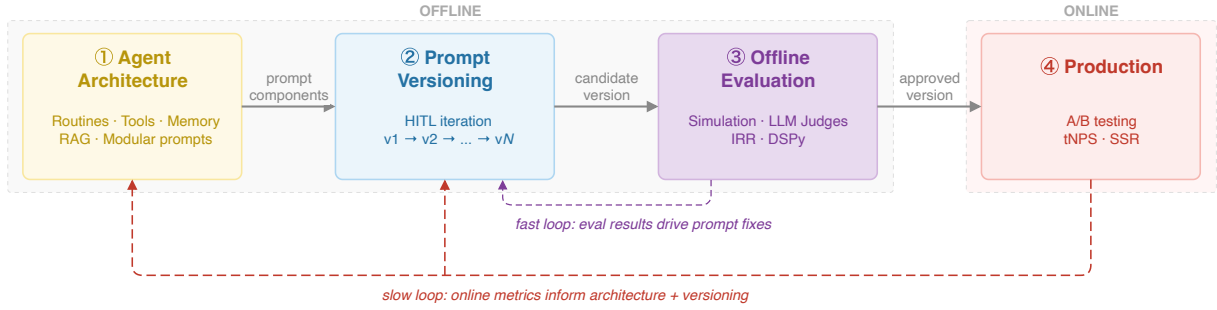


Figure 2: Overview of our evaluation-driven development framework. Four stages—context engineering, prompt iteration, offline evaluation, and production deployment—are connected by a feedback loop where online metrics inform subsequent iterations. The correlation between offline evaluation and production outcomes validates the pipeline’s predictive power.

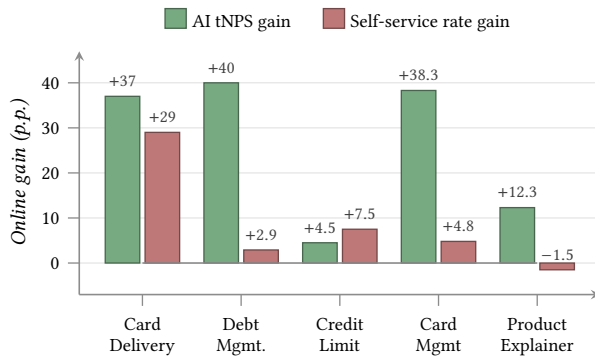


Figure 3: Online A/B-test gains for five production agents built with the same eval-driven framework. AI transactional NPS (tNPS) improves across all five domains; self-service rate improves in four (Product Explainer’s slight dip stems from transient LLM request failures). On most use cases the resulting AI tNPS lands within a few percentage points of expert human agents (Table 3).

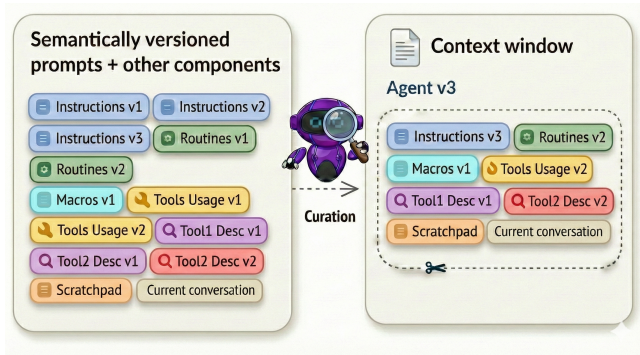


Figure 4: Our CS Agent context engineering is modular, and semantically versioned. Independently versioned components can be composed into distinct agent configurations.

(given user input) until it either completes the task, fails, or seeks

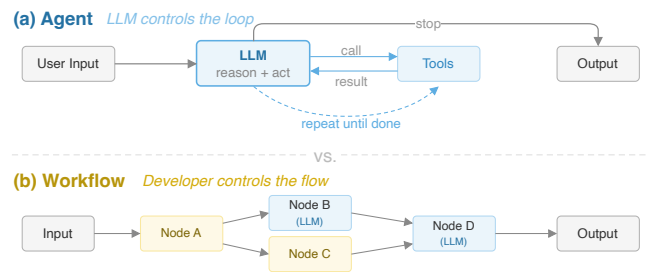


Figure 5: (a) Our CS agents follow the ReACT pattern: the LLM iteratively calls tools and reasons over results. (b) A workflow is a predefined DAG where the developer controls the execution flow; individual nodes may or may not use an LLM.

further user input. In contrast, we define *workflows* as predefined computation graphs or DAGs where the developer - and not the LLM - controls the execution flow. Each node in the graph may or may not invoke an LLM. **Figure 5** disambiguates between agents and workflows.

Frameworks like LangGraph [40] blur the boundaries between workflows and agents by allowing users to build graphs where some nodes are deterministic (workflow-like) and others involve autonomous decision-making (agent-like). In practice, however, our implementations adhere closely to the ReACT framework, with targeted modifications — such as guardrails and mandatory compliance checks — to address business and regulatory requirements.

3.2 Prompt Engineering for LLMs

LLMs generalize to new tasks when guided by well-crafted prompts: *in-context learning* supplies a few input–output examples without fine-tuning [5], and *chain-of-thought* prompting adds intermediate reasoning steps for complex tasks [22, 42]. *Modular prompting* decomposes tasks into coordinated stages—e.g., ReAct interleaves reasoning with tool-invoking actions [45].

3.3 Automated Prompt Optimization

Manual prompt engineering is labor-intensive and brittle, as small wording changes can substantially alter model behavior. *Automated prompt optimization* treats prompt text as an optimizable parameter, using search algorithms to maximize a target metric. Several methods cast this as LLM-driven search. APE [47] uses an LLM to generate and score candidate instructions. OPRO [44] iteratively proposes improved prompts conditioned on prior scores. ProTeGi [35] introduces natural-language “gradients” - critiques of failures - to guide beam-search refinement, while PromptBreeder [13] co-evolves task prompts and mutation operators. DSPy [21] provides a programmatic abstraction where LLM pipelines are defined as modular programs with typed signatures and compiled into optimized prompts. Within DSPy, GEPA [1] implements a reflection-based loop: a strong model critiques errors, synthesizes revisions, and selects candidates via Pareto-based trade-off analysis, optionally incorporating free-text human feedback.

3.4 Evaluation

Evaluating the quality of LLM-generated text is a prerequisite for any iterative improvement: without reliable evaluation, teams cannot distinguish signal from noise when comparing agents.

3.4.1 LLM-as-a-judge. Traditional evaluation of natural language generation often relied on reference-based metrics such as BLEU [34] and ROUGE [27], which approximate quality via surface-level overlap with references. However, for open-ended, multi-turn dialogue, these metrics correlate weakly with human judgment [28]. The *LLM-as-a-judge* paradigm instead uses a strong LLM to score or rank outputs against an explicit rubric [46]. Given a rubric, the judge model assesses whether a response meets quality dimensions such as correctness, helpfulness, safety, or adherence to instructions. This approach scales to thousands of examples, accommodates natural language variation, and can evaluate nuanced, domain-specific criteria that resist rule-based checking.

However, LLM judges exhibit systematic biases, including position and verbosity bias [46], self-enhancement bias [33], and sensitivity to rubric wording [4, 25, 43]. These failure modes motivate two strategies. First, judge ensembles — using multiple models or prompts — can mitigate individual biases, analogous to multi-annotator schemes [41]. Second, automated prompt optimization for evaluation criteria can systematically search for rubric formulations that maximize agreement with human annotations, reducing the brittleness of manually authored prompts.

3.4.2 Measuring reliability of judges. Whether evaluation is performed by humans, LLMs, or both, the fundamental question is how much we can trust the ratings. **Inter-rater reliability (IRR)** quantifies agreement among independent raters beyond chance. Cohen’s κ [7] assesses pairwise agreement on categorical labels:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (1)$$

where p_o is observed agreement and p_e is expected agreement under chance. Fleiss’s κ generalizes this to any number of raters [14], and Krippendorff’s α extends it to various data types and missing data [23]. High agreement (e.g., κ or α exceeding 0.8) instills confidence that quality labels are well-defined and consistently

applied. Cohen’s κ naturally extends to the LLM-as-a-judge setting, where two judge models take the role of two annotators. By comparing κ across judge pairs before and after prompt optimization, we evaluate whether GEPA improves rubric stability across model architectures.

3.4.3 Online Evaluation. Customer-support agents ultimately succeed or fail with real users, so online metrics from deployment are central to eval-driven development. Many teams track **CSAT (Customer Satisfaction)**, typically collected via post-interaction surveys on a Likert scale (e.g., “Were you satisfied with the help you received?”) [26]. Another widely used measure is **Net Promoter Score (NPS)**, which estimates how likely a user is to recommend the service [37]. For CS, organizations often use **Transactional NPS (tNPS)** - an NPS variant asked immediately after a support interaction. It is important to note we measure tNPS for AI Agents and human experts separately, using the support funnel described in Figure 1.

Beyond satisfaction, teams track effectiveness and deflection. **Self-service rate (SSR)** measures the fraction of support needs resolved via automated or self-serve channels without human escalation. High SSR and task success, together with strong CSAT and tNPS, indicate the agent is delivering effective support. It is worth calling out that there is usually a trade-off between SSR and tNPS, as you can typically improve overall tNPS by sacrificing SSR. One example is directly transferring hard cases or frustrated users to humans and only keeping the easier cases to be solved by the AI agent.

4 System Design

4.1 Design Principles

We propose the following design principles for CS AI agents:

- **Reliable.** Consistent behavior across interactions with graceful fallback to human agents, preserving full conversational context.
- **Correct.** Responses grounded in verified data from tools and knowledge bases — no hallucinated details or incorrect promises.
- **Delightful.** Empathetic, concise, and natural interactions reflecting the organization’s brand.
- **Compliant.** Adherence to regulations, data-handling policies, and internal business rules.
- **Measurable.** Quality quantifiable through offline evaluations and online business metrics (tNPS, SSR, task success rate).

4.2 Context Engineering for Agents

Prompt engineering—crafting textual instructions that steer LLM behavior—understates the scope of what a production system actually needs. For a CS agent, instructions are only one component occupying the context window. We adopt the framing of **context engineering**: the systematic design and assembly of the entire input presented to the model at every turn. **Figure 4** shows our context engineering design:

- **Instructions.** The agent’s role, definition, behavioral constraints, compliance guardrails, communication style, output specifications, and response construction guidelines. Instructions encode *how* the agent should behave.

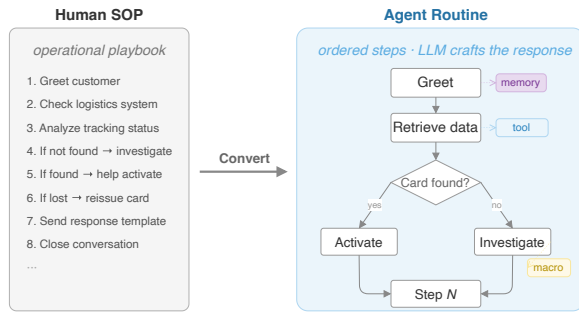


Figure 6: An example of a routine - translation of a human standard operating procedure (SOP) into ordered, agent-executable steps.

- **Routines.** Most customer support units within organizations issue standard operating procedures (SOPs) for human agents to assist customers using an automated process. We convert these SOPs into routines - ordered, step-by-step procedures that guide the agent through a domain-specific conversation flow (see **Figure 6**). For example, a card delivery investigation routine might prescribe: (1) greet and frame the case, (2) retrieve logistics data, (3) run address-type-specific investigation questions, (4) offer reissue if unresolved.
- **Macros.** Pre-written response templates for common scenarios — serve as modular building blocks that the agent adapts and condenses rather than composing responses from scratch while still allowing natural variation in phrasing.
- **Tool specifications and usage.** Each tool is described independently — name, input schema, expected output, and invocation conditions — so tool sets can be recombined across configurations. Orchestration rules then govern how tools are called in concert: parallel invocation strategies, transient error handling, confirmation gating (only confirm outcomes on explicit success), and grounding rules.
- **Working memory (scratchpad).** A persistent state object that tracks information gathered across the conversation: collected parameters, prior tool outputs, current routine step. The scratchpad prevents redundant tool calls while also enabling long-term personalization.

Each component is independently versioned, allowing a new agent configuration to be composed by selecting specific versions of each module (**Figure 4**). This transforms prompt iteration from monolithic rewrites into targeted, testable changes.

4.3 Infrastructure and Tool Design

A common failure mode in agent development is beginning with prompts and adding infrastructure later. Our experience indicates the opposite ordering is more effective: the data and action layers should be established first, because they constitute the durable foundation that persists across model generations.

4.3.1 Infrastructure layers. Our agent system rests on three infrastructure layers:

- **Identity and authentication.** Secure, consent-aware access to customer data.
- **Action APIs.** Idempotent, transactional endpoints that execute real business logic (e.g., reissuing a card, adjusting a payment plan). Idempotency is critical: because LLMs may retry tool calls on transient failures or ambiguous results.
- **Audit trails.** Comprehensive logging of every tool call—inputs, outputs, latency, and errors—with masking of sensitive fields, strict access controls, and retention schedules aligned with legal and regulatory requirements—for compliance, debugging, and post-hoc analysis.

4.3.2 Tool design principles. Tool design directly affects agent reliability:

Compose deterministic flows in code, not in prompts. When a task requires chaining multiple data sources—for example, checking carrier tracking status *and* querying the internal order management system—we wrap the orchestration into a single composite tool rather than relying on the LLM to sequence two separate calls. This reduces latency, eliminates sequential-dependency errors where the model calls tools in the wrong order or drops an intermediate result, and reserves the LLM’s reasoning capacity for tasks that genuinely require judgment.

Tool descriptions are prompts. The schema and docstring of each tool function as part of the agent’s prompt context (§4.2). Ambiguous or incomplete tool descriptions lead to misuse—incorrect parameters, calls at the wrong time, or misinterpretation of outputs. We treat tool documentation with the same rigor as other prompt components: each tool includes a natural-language description, a typed input schema, the expected output format, example invocations, and explicit guidance on when *not* to call it.

Minimize output surface area. Tools return only the fields the LLM needs to reason about, not the full upstream API response. Raw responses often contain dozens of internal fields that consume context-window budget and increase the risk of the model latching onto irrelevant details.

Design for idempotency. Because LLMs may retry tool calls on transient failures or ambiguous results, every action tool must be safely re-callable without side effects.

4.4 Development Lifecycle

Figure 2 depicts our development process using two nested iteration loops.

The pipeline flows through four stages. First, *Agent Architecture* (①) defines the modular prompt structure that constitutes the agent’s context (§4.2). These components are passed to *Prompt Versioning* (②), where domain experts iterate on individual modules, producing a sequence of candidate versions ($v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_N$). Each candidate is then evaluated in *Offline Evaluation* (③) using real and/or simulated conversations scored by calibrated LLM judges. Only versions that pass offline evaluation—demonstrating improved or stable metrics relative to the current baseline—are promoted to *Production* (④), where they are deployed via A/B tests and measured on tNPS, SSR, and other business metrics.

Two feedback loops drive continuous improvement:

- **Fast loop** Offline evaluation results feed directly back into prompt versioning. When an evaluator flags a failure—for example, the agent omitting address verification—the team traces the failure to the responsible prompt component, applies a targeted fix, and re-evaluates. Fixes are surgical: modifying one routine step does not require re-testing the entire prompt.
- **Slow loop** Production metrics reveal patterns that offline evaluation may not capture—for instance, a consistent tNPS gap on a specific customer segment, or a rising escalation rate on a newly launched product. These signals inform architectural changes: adding new routines, redesigning tool interfaces, or restructuring the context allocation.

This two-loop structure helps prove that evaluation quality determines iteration velocity. A reliable fast loop—enabled by calibrated judges and rigorous inter-rater agreement (§3.4)—allows teams to iterate confidently without waiting for slow, sparse online signals.

4.5 Deployment Strategy

Deploying an agent to production at scale requires a progressive rollout strategy that balances iteration velocity with risk management.

4.5.1 Progressive rollout. New agent versions are deployed at a small percentage (1–5%) while monitoring key metrics (SSR, tNPS, escalation rate, error rate).

4.5.2 Confidence-based escalation. The agent operates within a human-in-the-loop architecture (Figure 1). When the agent encounters situations outside its defined routines, fails to retrieve the necessary data, or detects low confidence in its proposed response, it transfers the conversation to a human agent.

4.5.3 Model portability. Because foundation models evolve rapidly, our system is designed to accommodate model transitions with minimal disruption. The modular context engineering architecture (§4.2) enables targeted prompt adjustments. We maintain per-model evaluation baselines and run the full evaluation suite before any model migration, treating model upgrades as first-class prompt changes subject to the same eval-driven process described in subsequent sections. More details using a case study in Section 5

5 Building a CS Agent at Scale - A Case Study

We validate our framework on a **card delivery** agent — one of the highest-volume intents at our organization, handling “Where is my card?” inquiries. It exercises the full framework: it requires multi-step reasoning over real-time logistics data, context-dependent branching (e.g., address type determines the investigation path), and careful grounding to avoid misstating delivery timelines.

5.1 Agent Overview

The card delivery agent’s mission is to resolve card status inquiries. A customer contacts support after ordering a new card (due to first issuance, loss, theft, damage, or expiration), and the agent must determine the card’s current status and guide the customer to resolution.

Routine. The agent follows a routine (see Figure 6 for the general pattern) with the following steps: (1) greet and frame the case, (2)

retrieve logistics and tracking data via tools, (3) analyze delivery status against expected timelines, (4) run address-type-specific investigation questions (e.g., apartment access, concierge, PO box), and (5) offer card reissue if the issue cannot be resolved.

Tools. The agent has access to a small, domain-specific toolset:

- `get_customer_profile` — retrieves identity, address, and account details for the authenticated customer.
- `check_card_delivery` — queries the order management system for tracking status, expected delivery date, and delivery confirmation.
- `reissue_card` — initiates a card reissue to the same or a new address.

5.2 Building Evals

We now discuss how we built *LLM-as-a-judge* evaluators for this use case. Using rigorous evals, we A/B tested 11 variants of the agent online, leading to large gains (see Results for details).

Data Collection and Annotation. To rigorously evaluate the performance of the agent scenarios, we constructed a dataset spanning five evaluation categories (Evals E1–E5), see Table 1 for details. This and all other datasets used in this work are privacy-preserving datasets derived from real customer-support conversations through the data-minimization and pseudonymization pipeline described in Section 6.5. Ground-truth binary labels were obtained through a multi-annotator procedure involving three operations analysts with specialized domain knowledge of the target business logic. For each sample, analysts provided both a classification label and a detailed textual explanation justifying their decision. The final ground truth for each sample was determined by majority vote among the three annotators. The accompanying textual explanations were retained as semantic feedback signals for the subsequent optimization phase. **It is important to note that only a few hundred high quality binary labels are enough to build an effective eval.**

Description	# Count	# Class	Freq.
			Min. Class
E1: Agent Reissue Failure	888	2	0.49
E2: Customer Input Verification	90	2	0.22
E3: Card Delivery Data Check	816	2	0.43
E4: Response Conciseness	246	2	0.49
E5: Resolution Completeness	90	2	0.49

Table 1: Summary statistics of the evaluation datasets.

Prompt Optimization. We used GEPA [1] within the DSPy framework to automatically refine the eval instructions.

All classification evaluations used GPT-4.1-mini as the base model and GPT-5.1 as a reflection model to critique errors, synthesize revisions, and propose semantically grounded prompt edits informed by analyst feedback. We ran GEPA with `auto=light` (approximately 500 iterations), reflection minibatch size = 3, and Pareto-based candidate selection to balance trade-offs across optimization criteria. We also provided the optimizer with the free-text rationales collected during annotation, enabling the reflection model to anchor prompt updates in expert reasoning rather than surface heuristics.

We compared against (i) a *majority-class* baseline that always predicts the most frequent label, and (ii) a *starter-prompt* baseline consisting of a brief, manually written instruction set. We repeated each experiment five times and report mean accuracy and 95% confidence interval across runs.

We stratified the annotated dataset into train, validation, and test splits using a 40%/30%/30% ratio. During the optimization phase, GEPA utilized the train and validation set to iteratively refine prompt candidates based on the provided feedback. Final performance results reported were measured on the held-out test set.

Inter-rater Agreement Analysis. To validate the robustness of evals after optimization, we analyzed the consistency of predictions across several state-of-the-art language models: GPT-4o, GPT-4o-mini, GPT-4.1, GPT-4.1-mini, GPT-5, o3, o3-mini, all operated under data-processing agreements in controlled environments. The model temperature was set to 0, and all other hyperparameters were left at their default values. This analysis was conducted in two phases: first, using the manual starter prompt and subsequently using the optimized prompt.

We quantified agreement between model pairs using Cohen’s kappa (κ) as described in the background section. By comparing κ before and after prompt optimization, we evaluate whether GEPA improves the stability of the rubric such that it induces more consistent judgments across different model architectures.

6 Offline and Online Results

6.1 Prompt Optimization

Eval	Accuracy (%)		
	Baseline	Starter Prompt	Optimized Prompt
E1	51.11	77.78 $\pm$ 0.49	82.00 $\pm$ 0.09
E2	77.78	68.88 $\pm$ 3.41	88.89 $\pm$ 0.00
E3	56.62	58.07 $\pm$ 0.26	73.01 $\pm$ 0.76
E4	51.11	74.00 $\pm$ 0.95	76.89 $\pm$ 0.79

Table 2: Evaluator accuracy (mean with 95% confidence intervals in subscript) before and after prompt optimization.

Table 2 summarizes the performance and demonstrates that automated optimization consistently outperforms both the majority-class baseline and manual prompt engineering across all tasks. Notably, in E3, the optimization delivered a substantial improvement, raising accuracy from a manual baseline of 58.07 to 73.01. This illustrates the framework’s capability to capture complex evaluation criteria where manual prompting failed to align with expert consensus. Furthermore, the narrow confidence intervals observed across all optimized prompts (ranging from 0.00 to 0.79) indicate that the GEPA optimization process yields robust prompts that are consistent across multiple inference runs. **A sample before-and-after prompt for Eval E2 can be found in Appendix A.**

A critical insight emerged from E2, a binary evaluator characterized by a high class imbalance (baseline accuracy of 77.78). In this instance, the manual starter prompt degraded performance to 68.88, performing worse than the majority-class baseline. **This**

highlights the risks inherent in manual prompt engineering, where human-written instructions may inadvertently introduce noise or bias that contradicts the underlying task distribution. Conversely, optimization successfully recovered performance, achieving an accuracy of 88.89 by aligning the GPT-4.1-mini base model with the rigorous logic of the human specialists.

For tasks with more balanced class distributions, specifically Eval E1 and E4, where baselines were approximately 51.11, the Optimized Prompt demonstrated high reliability, achieving 82.00 and 76.89 accuracy, respectively. However, the positive lift from the starter prompt was small compared to E2.

6.2 Inter-rater Agreement

Figures 7a and 7b present the pairwise Cohen’s Kappa coefficients for Eval E1, calculated before and after GEPA prompt optimization, respectively.

The starter prompt resulted in negligible agreement across the majority of model pairs. For instance, the interaction between the GPT-4.1 and GPT-5 families yielded a low coefficient ($\kappa = 0.109$). Notably, the agreement between GPT-4.1 and GPT-4.1-mini was 0.00. Inspection of the predictions revealed that the unoptimized GPT-4.1-mini assigned a single class to all test instances, resulting in zero variance and a consequent κ of zero. This lack of consensus suggests that the manual starter prompts were ambiguous, leading the models to rely on internal priors.

Conversely, the GEPA-optimized prompts led to a substantial convergence in judge behavior. Post-optimization, the agreement between GPT-4.1 and GPT-4.1-mini increased to $\kappa = 0.745$, while the agreement between GPT-4.1 and GPT-4o reached $\kappa = 0.895$. The highest consensus was observed between the reasoning-intensive models, with GPT-5 and o3 achieving near-perfect alignment $\kappa = 0.950$. These results indicate that the optimized prompts effectively grounded the evaluation criteria, rendering the judging process largely invariant to the specific model architecture.

6.3 Online Experiments

We used the aforementioned evals to deploy 10 different variants of the card-delivery use case agent and compared them against our legacy customer support solution across a large number of real customer interactions.

Each variant was launched at 1% traffic, and subsequently increased to larger percentages. The evaluator scores were used to exclude non-promising variants. The baseline variant (V0) was limited to a knowledge-base retrieval tool and a human-transfer tool, unable to perform domain actions. Subsequent versions progressively added: a structured routine derived from SOPs; domain-specific tools for logistics tracking and customer data retrieval; frustration detection and proactive escalation; self-contained card reissue capability, eliminating the need for human handoff on the most common resolution path; tool-selection hierarchies and conciseness budgets to curb verbosity and tool misuse — each change driven by failures surfaced through the offline evals.

6.3.1 Business Metric Impact. The AI agent delivered a 37 percentage point improvement in AI tNPS, reflecting substantially higher customer satisfaction. Additionally, we observed a 29 p.p increase in SSR, indicating that more customers successfully resolved their

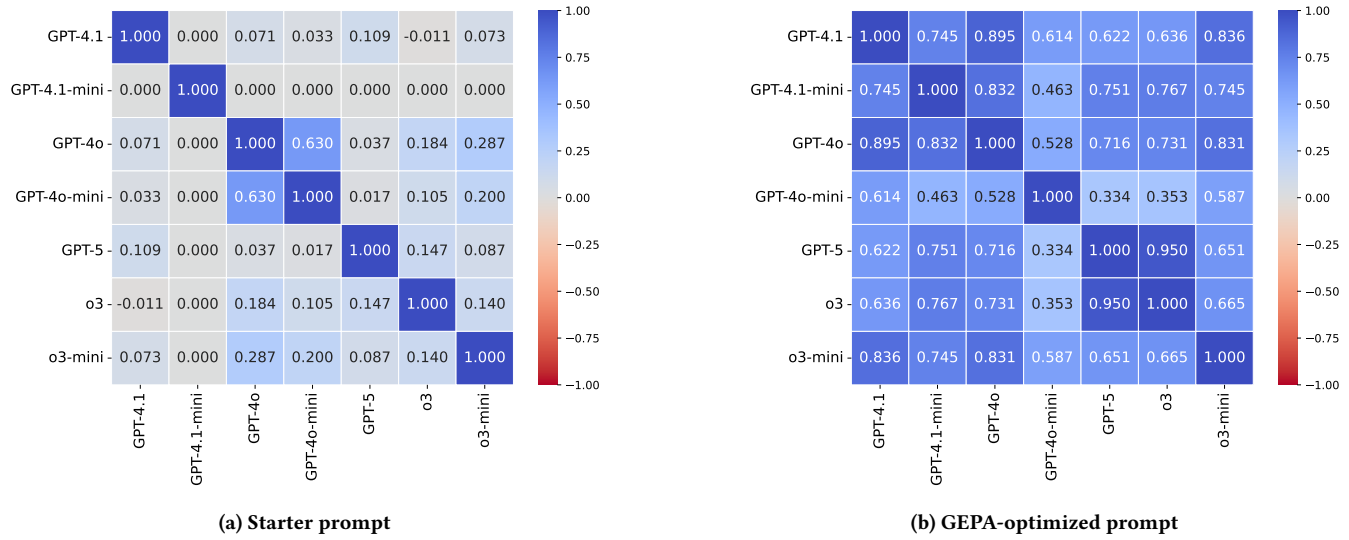


Figure 7: Eval E1 - Pairwise Cohen's κ heatmaps among seven language models, comparing the starter prompt (left) and the GEPA-optimized prompt (right).

issues without escalation to human agents. These gains translate directly to improved customer experience and reduced operational costs at scale. Notably, the AI tNPS came within 10 p.p. of the expert human-agent tNPS score.

6.3.2 Model Ablation. We evaluated the impact of the underlying base model by comparing agent variants built on GPT-4.1 versus GPT-5 (these constitute 2 of the 10 variants we tried). Notably, transitioning to GPT-5 required prompt adaptation: GPT-5 exhibits stricter instruction-following behavior compared to GPT-4.1, which would implicitly infer unstated requirements. For instance, GPT-5 failed to call contextual tools before responding unless explicitly instructed, whereas GPT-4.1 inferred this requirement from context. To address this, we introduced a *tool-first principle* in the Instructions component - a single directive mandating tool calls before composing responses when customer-specific data is required. This minimal intervention restored the desired behavior.

Figure 8 shows the eval failure rates for both variants for each binary evaluator with 95% confidence intervals. The GPT-5-based agent achieved reductions across all evaluation categories, except E4 (GPT5 was generating more verbose answers). **Crucially, the GPT-5 variant resulted in better online metrics, correlating with its offline eval improvements when compared to the GPT-4.1 variant.**

These results highlight a key insight: more capable models with stricter instruction-following require more explicit prompts but can deliver superior performance when properly guided. The modular prompt architecture enabled rapid, targeted modifications.

6.3.3 Offline-Online Correlation. A core claim of our framework is that rigorous offline evaluation enables predictive iteration: improvements in simulation metrics should correlate with improvements in production outcomes. Figure 9 validates this hypothesis by plotting the relationship between offline evaluation performance and online tNPS across multiple agent versions (V0–V10). Each

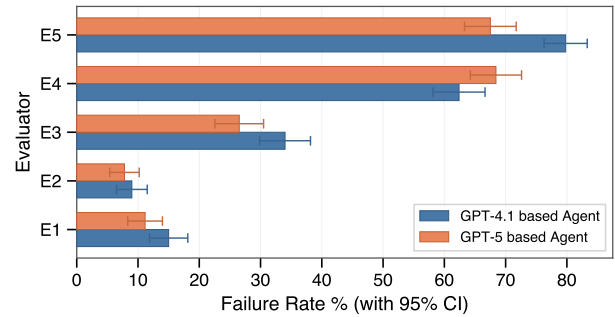


Figure 8: Evaluation failure rates for GPT-4.1 and GPT-5 with 95% confidence intervals. GPT-5 almost always achieves lower failure rates.

point represents a deployed agent variant; the x -axis shows the change in average evaluation failure rate relative to the baseline (V0), while the y -axis shows the corresponding change in tNPS.

We observe a strong positive correlation: versions with lower eval failure rates consistently achieve higher tNPS scores. Notably, versions V10 and V8 achieved the largest offline improvements (approximately 20 - 25 p.p. reductions in failure rate) and correspondingly demonstrated the highest tNPS gains (37 p.p. relative to V0). Conversely, V6, which showed minimal offline improvement, delivered near-zero tNPS lift.

This correlation validates our central thesis: investment in evaluation quality - through rigorous annotation, GEPA-optimized judge prompts, and multi-model agreement analysis enables teams to iterate confidently offline, knowing that improvements will translate to online positive gains. The framework reduces the need for expensive online experimentation, enabling fast iteration cycles.

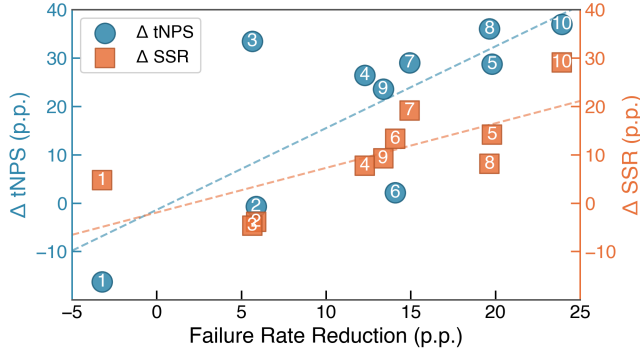


Figure 9: Offline evaluation improvements are positively correlated with online gains in both tNPS and SSR across deployed agent variants (numbered by version), measured relative to the V0 baseline.

6.4 Generalization Across Production Use Cases

To assess whether the same eval-driven workflow generalizes to materially different customer-support domains, we report results from 4 additional agents. We describe two in detail below and summarize all five deployments in Table 3.

6.4.1 Debt Management Agent. This agent helps customers regularize overdue personal-loan debt by detecting overdue status, simulating offers across installment plans, and guiding the customer. This use case requires multi-step numerical reasoning over financial simulations and empathetic communication with customers. Using the same eval-driven workflow, the best-performing variant substantially improved customer-perceived quality - lifting AI tNPS by **40 p.p.** - while maintaining and slightly improving automation (self-service rate and return rate both moved favorably) and reducing evaluator failures by **3.5 p.p.** (Table 3).

6.4.2 Credit Limit Agent. This agent handles credit-card limit questions by explaining limit breakdowns, clarifying discrepancies, and recommending alternative credit options when an increase is unavailable. Unlike card delivery, this use case requires precise grounding across multiple financial data sources and careful communication around decisions. We built three domain-specific evaluators and followed the same iteration cycle. Relative to control, the deployed agent improved AI tNPS, self-service rate, and resolution rate, and sharply cut its evaluator failure rates—for example, account-specific-context and customer-empathy failures dropped by **14.95** and **11.32 p.p.**, respectively (Table 3).

6.4.3 Cross-Use-Case Summary. Table 3 summarizes the online results across the five production deployments.

Taken together, these use cases span distinct domains - from logistics to debt management, credit management, card servicing, and product education - with different routines, tools, and evaluation rubrics. The same framework yields consistent customer-satisfaction (tNPS) gains across all five. The Δ Human tNPS column further shows that, for four of the five agents, the AI reaches within ~ 1 – 10 p.p. of expert human-agent tNPS, with the largest remaining gap in the hardest domain (debt management, -23.6 p.p.).

Use Case	tNPS Gain	SSR Gain	Δ Human tNPS
Card Delivery	+37	+29	−10
Debt Management	+40	+2.9	−23.6
Credit Limit	+4.5	+7.5	−7.1
Card Management	+38.3	+4.8	−1.1
Product Explainer	+12.3	−1.5 [‡]	−6.2

Table 3: Online A/B-test results across five production agents. All results are in percentage points (p.p.); Δ Human tNPS is the AI agent’s tNPS minus the expert human-agent tNPS (negative = below human). All five agents also showed substantial reductions in evaluator failure rates. [‡]The Product Explainer SSR dip reflects transient LLM request failures.

6.5 Responsible Deployment: Data Protection and Privacy by Design

Our agents operate in a highly regulated financial environment under strict privacy-by-design and privacy-by-default principles. All datasets used are derived from customer-support interactions through a data-minimization and pseudonymization pipeline that removes or masks direct identifiers and redacts sensitive financial fields. Access to conversation logs and evaluation datasets is role-based, audited, and time-limited.

When we use third-party foundation models (e.g., for judge ensembles or prompt optimization), we do so under Data Processing Agreements (DPAs) that contractually prohibit training on Nubank data, with requests restricted to minimized, privacy-preserving inputs and cross-border processing governed by an international transfer framework.

Finally, the agents make no unconstrained or irrevocable decisions about credit, debt management, or limits; they operate inside existing regulated processes subject to established policies, human oversight, and channels for contestation and review. Logging and audit trails apply the same safeguards without compromising data protection or customer trust.

7 Conclusion

We presented an eval-driven methodology for building high-quality customer-support agents. The approach couples a maintainable, modular prompt architecture with rigorous eval design (including inter-rater reliability) and automated judge-prompt optimization via GEPA. In both offline simulation and online A/B tests, this workflow enabled rapid iteration while preserving a clear quality bar, yielding material gains in customer satisfaction and self-service over a legacy support system. The same framework delivered consistent online customer-satisfaction gains across five deployed agents spanning logistics, debt management, credit-limit support, card management, and product explanation, suggesting that the methodology generalizes across customer-support domains. Future work will broaden use case coverage, generalize to different open-source or proprietary models, and close the loop on fine-tuning by using the same eval-driven methodology to inform data curation and training objectives, applying strict privacy and data-minimization controls.

References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soyulu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [2] Anthropic. Building effective agents, 2024. URL <https://www.anthropic.com/engineering/building-effective-agents>. Published Dec 19, 2024. Accessed: 2026-02-02.
- [3] Anysphere, Inc. Cursor: The ai code editor, 2024. URL <https://cursor.com>. Accessed: 2025-02-05.
- [4] Negar Arabzadeh and Charles L. A. Clarke. A human-ai comparative analysis of prompt sensitivity in LLM-based relevance judgment, 2025. URL <https://arxiv.org/abs/2504.12408>. Related DOI: 10.1145/3726302.3730159.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. Reinforcement learning for long-horizon interactive LLM agents. *arXiv preprint arXiv:2502.01600*, 2025.
- [7] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.
- [8] Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- [9] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [10] Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*, 2025.
- [11] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, et al. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.
- [12] Yue Feng, Yunlong Jiao, Animesh Prasad, Nikolaos Aletras, Emine Yilmaz, and Gabriella Kazai. Schema-guided user satisfaction modeling for task-oriented dialogues. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2079–2091, Toronto, Canada, 2023. Association for Computational Linguistics.
- [13] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- [14] Joseph L. Fleiss. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5):378–382, 1971.
- [15] Asbjørn Følstad, Effie L.-C. Law, and Nena van As. Conversational breakdown in a customer service chatbot: Impact of task order and criticality on user trust and emotion. *ACM Transactions on Computer-Human Interaction*, 31(5), 2024. doi: 10.1145/3690383.
- [16] Google. Google antigravity: Experience liftoff with the next-generation ide, 2025. URL <https://antigravity.google/>.
- [17] James Gung, Emily Moeng, Wesley Rose, Arshit Gupta, Yi Zhang, and Saab Mansour. NatCS: Eliciting natural customer support dialogues. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 9652–9677, Toronto, Canada, 2023. Association for Computational Linguistics.
- [18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shitong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- [19] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *arXiv e-prints*, 2023. doi: 10.48550/arXiv.2310.06825. URL <https://arxiv.org/abs/2310.06825>.
- [20] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan O. Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- [21] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhaman A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html. Conference paper.
- [22] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 22199–22213, 2022.
- [23] Klaus Krippendorff. Reliability in content analysis: Some common misconceptions and recommendations. *Human Communication Research*, 30(3):411–433, 2004.
- [24] Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long tasks. *arXiv preprint arXiv:2503.14499*, 2025.
- [25] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2757–2791, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.138. URL <https://aclanthology.org/2025.emnlp-main.138/>.
- [26] Rensis Likert. A technique for the measurement of attitudes. *Archives of Psychology*, 22(140):1–55, 1932.
- [27] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics. URL <https://aclanthology.org/W04-1013/>.
- [28] Chia-Wei Liu, Ryan Lowe, Iulian Serban, Mike Noseworthy, Laurent Charlin, and Joelle Pineau. How NOT to evaluate your dialogue system: An empirical study of unsupervised evaluation metrics for dialogue response generation. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2122–2132, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1230. URL <https://aclanthology.org/D16-1230/>.
- [29] John Mendonça, Patrícia Pereira, Miguel Menezes, Vera Cabarrão, Ana C. Farinha, Helena Moniz, Alon Lavie, and Isabel Trancoso. Dialogue quality and emotion annotations for customer support conversations. In *Proceedings of the 3rd Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, pages 9–21, Singapore, 2023. Association for Computational Linguistics.
- [30] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Toronto, Canada, 2025. ACM.
- [31] Stephen Obadinma, Faiza Khan Khattak, Shirley Wang, Tania Sidhom, Elaine Lau, Sean Robertson, Jingcheng Niu, Winnie Au, Alif Munim, Karthik Raja K. Bhaskar, Bencheng Wei, Iris Ren, Waqar Muhammad, Erin Li, Bukola Ishola, Michael Wang, Griffin Tanner, Yu-Jia Shiah, Sean X. Zhang, Kwesi P. Apponsah, Kanishk Patel, Jaswinder Narain, Deval Pandya, Xiaodan Zhu, Frank Rudzicz, and Elham Dolatabadi. Bringing the state-of-the-art to customers: A neural agent assistant framework for customer service support. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 440–450, Abu Dhabi, UAE, 2022. Association for Computational Linguistics.
- [32] OpenAI. Introducing deep research, February 2025. URL <https://openai.com/index/introducing-deep-research/>. Accessed: 2025-02-05.
- [33] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-2197. URL https://papers.nips.cc/paper_files/paper/2024/hash/7f1f0218e45f5414c79c0679633e47bc-Abstract-Conference.html.
- [34] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In Pierre Isabelle, Eugene Charniak, and Dekang Lin, editors, *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics. doi: 10.3115/1073083.1073135. URL <https://aclanthology.org/P02-1040/>.
- [35] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search. *arXiv preprint arXiv:2305.03495*, 2023.
- [36] Cheng Qian, Haode Qi, Gengyu Wang, Ladislav Kunc, and Saloni Potdar. Distinguish sense from nonsense: Out-of-scope detection for virtual assistants. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 502–511, Abu Dhabi, UAE, 2022. Association for Computational Linguistics.
- [37] Frederick F. Reichheld. The one number you need to grow. *Harvard Business Review*, 81(12):46–54, 2003.
- [38] Philipp Reinhard, Mahe Manhai Li, Christoph Peters, and Jan Marco Leimeister. Generative ai in customer support services: A framework for augmenting the routines of frontline service employees. In *Proceedings of the 57th Hawaii International Conference on System Sciences (HICSS)*, pages 468–477. ScholarSpace, 2024.
- [39] Hanchen Su, Wei Luo, Yashar Mehdad, Wei Han, Elaine Liu, Wayne Zhang, Mia Zhao, and Joy Zhang. LLM-friendly knowledge representation for customer

support. In *Proceedings of the 31st International Conference on Computational Linguistics: Industry Track*, pages 496–504, Abu Dhabi, UAE, 2025. Association for Computational Linguistics.

- [40] LangChain Team. Langgraph: A library for orchestrating multi-agent systems, 2024. URL <https://github.com/langchain-ai/langgraph>. Accessed: 2026-02-06.
- [41] Pat Verga, Sebastian Hofstätter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models, April 2024. URL <https://arxiv.org/abs/2404.18796>.
- [42] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837, 2022.
- [43] Yusuke Yamauchi, Taro Yano, and Masafumi Oyamada. An empirical study of LLM-as-a-judge: How design choices impact evaluation reliability, 2025. URL <https://arxiv.org/abs/2506.13639>.
- [44] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2023.
- [45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [46] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [47] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitit, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *The eleventh international conference on learning representations*, 2022.
- [48] Jie Zhu, Huaixia Dou, Junhui Li, Lifan Guo, Feng Chen, Chi Zhang, and Fang Kong. Evaluating, synthesizing, and enhancing for customer support conversation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. To appear.

A Evaluator Prompt Evolution

This appendix presents a before-and-after comparison of an evaluator prompt from the card delivery use case. We show the *starter prompt* (Table 2, “Starter Prompt”) and the *GEPA-optimized prompt* (Table 2, “Optimized Prompt”).

A.1 Task Description

The **Input Verification** evaluator assesses whether the AI assistant correctly collected the customer’s delivery address during a card reissuance conversation. The assistant must gather *both* core address fields (street, number, city, state, postal code) *and* at least one supplementary detail (e.g., apartment number, building name, landmark). The evaluator returns:

- **Score 0 (pass):** Sufficient address information gathered, or gathering was not required.
- **Score 1 (fail):** The assistant failed to collect the necessary address details.

A.2 Starter Prompt (Before Optimization)

The starter prompt is intentionally brief (~12 lines) and relies on commonsense reasoning for edge cases.

```
You are an automatic evaluator for customer-support chat logs.

TASK
-----
Look at the complete conversation between USER and ASSISTANT.
- If the ASSISTANT ever asks the USER for BOTH:
  1. the address itself (street, city, zip, etc.) AND
  2. at least one extra detail (e.g., apartment number, suite,
    floor,
    building name, landmark, delivery instructions),
  then return: {"score": 0, "analysis": "<why it passed>"}.
  Otherwise return: {"score": 1, "analysis": "<why it failed>"}.

RULES
-----
- Base your decision on the *entire* conversation.
- Respond with valid JSON only, no additional text.

Output (JSON)
{ "score": 0/1, "analysis": "<why it passed or failed>." }
```

```
- Otherwise return: {"score": 1, "analysis": "<why it failed>"}.

RULES
-----
- Base your decision on the *entire* conversation.
- Respond with valid JSON only, no additional text.

Output (JSON)
{ "score": 0/1, "analysis": "<why it passed or failed>." }
```

```

# Conversation history
{messages}
```

Listing 1: Starter prompt (v1_0) for the Input Gathering evaluator. The rubric is concise but lacks guidance on edge cases such as conversations that do not involve card reissuance, customers who confirm sending to the same address on file, and the distinction between initial issuance and reissuance.

Observed failure modes. Annotation and inter-rater analysis revealed four systematic errors with the starter prompt:

- (1) **False positives on non-reissuance chats.** No distinction between reissuance conversations (where address gathering matters) and tracking/status inquiries.
- (2) **Missed “same-address” shortcut.** When a customer confirmed the card should go to the address on file, the evaluator still penalized the assistant for not explicitly gathering fields.
- (3) **Ambiguous “extra detail.”** No definition of what qualifies as an extra detail in the target region (e.g., whether “address type” such as residential/commercial counts).

A.3 GEPA-Optimized Prompt (After Optimization)

The optimized prompt was produced by GEPA [1] within the DSPy framework (~500 iterations, GPT-5.1 as reflection model, human-annotator rationales as semantic feedback). It addresses each failure mode above with explicit decision logic.

```
You are an automatic evaluator for customer-support chat logs
about payment card issues.
Your job is to read the entire conversation and output a JSON
object with a score and a
brief analysis, following the exact logic below.
```

```
Input format and scope
```

```
- Conversation history has messages labeled by role ([user], [
assistant],
[human-assistant], [system], [upstream-flows]).
- Only evaluate [user]-[assistant] messages; ignore all other
roles.
```

```
What to evaluate
```

```
Determine whether the conversation involves card reissuance with
delivery and whether
the assistant correctly handled address collection when needed.
```

```
Key definitions and address details
```

```
- Core address fields: Street/thoroughfare AND Number; City;
State; ZIP/postal code.
(Neighborhood useful but not strictly required if other core
fields are present.)
- Extra delivery-relevant detail (need >=1): apartment/suite/unit
, block, floor,
building name, complement, business name, landmark, or delivery
instructions.
```

```

"Address type" (Residential/Commercial) does NOT count.

(...) "Important clarifications" on virtual cards, same-address-on-
-file, initial
issuance vs. reissue, and verification steps elided; see Table 4
...)

Decision steps
1) Does the user need to reissue their card? Triggers: lost/
stolen/damaged/expired/
not-received with a request for a new card; explicit reissue/
second-copy request;
or assistant proposes reissue and the flow proceeds on that
basis.
- If NO -> {"score": 0, "analysis": "Input gathering not
required because card
reissuance was not needed."}
2) If reissue is needed: is a physical card actually being sent (
delivery involved)?
- If NO -> {"score": 0, "analysis": "No card delivery involved
..."}
3) If reissue with delivery: will the card go to the same address
on file?
- If user explicitly confirms same address -> {"score": 0, "
analysis": "Address
confirmation is sufficient..."}
4) Otherwise (new/changed address needed): evaluate whether the
ASSISTANT handled
address gathering correctly.
- PASS (0): assistant explicitly requested all core fields (
incl. ZIP and number)
AND >=1 extra detail; OR user provided all core fields AND
>=1 extra detail.
- FAIL (1): vague "full/complete address" without enumerating
core fields; OR no
extra detail requested; OR proceeded without same-address
confirmation or a
proper explicit request; OR relied on [human-assistant] to
gather details.
(...) detailed Pass/Fail sub-clauses and "Guidance distilled
from prior cases"
elided; see Table 4 ...)

Output format
- Return valid JSON only, with exactly two keys: "score" (0 or 1)
and "analysis".
- No additional text, no markdown.

# Conversation history
{messages}

```

Listing 2: GEPA-optimized prompt (v1_4) for the Input Gathering evaluator, *abridged* for space (the full prompt grows to ~80 lines). It adds role scoping, domain-specific address definitions, a four-step decision tree, and guidance distilled from annotator feedback; elided blocks are summarized in Table 4.

A.4 Key Differences

Table 4 summarizes the principal differences between the two prompts.

Table 4: Structural comparison: starter vs. GEPA-optimized prompt for the Input Gathering evaluator.

Aspect	Starter (v1_0)	Optimized (v1_4)
Length	~12 lines	~80 lines
Role scoping	None; evaluates all message roles	Explicit: only [user]–[assistant]; ignores [human-assistant], [system], [upstream-flows]
Reissuance gathering	Not addressed; runs on all conversations	Four-step decision tree: (1) reissue needed? (2) physical delivery? (3) same-address confirmation? (4) evaluate gathering
Address definitions	Generic: “street, city, zip, etc.”	Domain-specific core fields (street+number, city, state, ZIP) and extra details (apartment, block, complement, landmark); excludes “address type”
Edge-case handling	None	Covers virtual-card-only, initial issuance vs. reissue, same-address confirmation, vague “send complete address” requests, verification steps
Annotator feedback	Not incorporated	“Guidance distilled from prior cases” derived from human rationales supplied to GEPA

A.5 Impact

As reported in Section 6.1 (Table 2), the GEPA-optimized prompt yielded measurable accuracy improvements on the held-out test set. The structured decision tree reduced both false positives (flagging non-reissuance conversations) and false negatives (missing genuine address-gathering failures). Inter-rater agreement (Section 6.2) confirmed that the optimized prompt produced substantially more consistent judgments across seven language models, with pairwise Cohen’s κ rising from near-zero to $\kappa > 0.7$ for most model pairs.